附 录 A：检索增强生成部分源程序

```
import sys
import os

# 获取当前文件所在目录（backend）
current_dir = os.path.dirname(os.path.abspath(__file__))

# 获取项目根目录
project_root = os.path.dirname(current_dir)

# 将两个目录都添加到 Python 路径
sys.path.insert(0, project_root)  # 项目根目录（用于 backend.src.xxx 导
入）
sys.path.insert(0, current_dir)   # backend 目录（用于 src.xxx 导入）

# 继续原有的导入

from typing import Annotated, Dict

from typing import Annotated, Dict

from fastapi import File, UploadFile, Form, Body
from fastapi import FastAPI, Request
from fastapi_health import health
from fastapi.middleware.cors import CORSMiddleware
from volcenginesdkarkruntime import Ark

from src.llm_api_request import ChatRequest
from src.main import *
# from src.QA_integration import *
from src.QA_integration_new import *
from src.shared.common_fn import *
import uvicorn
import asyncio
import base64
from langserve import add_routes
from langchain_google_vertexai import ChatVertexAI
from src.api_response import create_api_response
from src.graphDB_dataAccess import graphDBdataAccess
from src.graph_query import get_graph_results
from src.chunkid_entities import get_entities_from_chunkids
```

```python
from src.post_processing import create_fulltext,
create_entity_embedding
from sse_starlette.sse import EventSourceResponse
import json
from starlette.middleware.sessions import SessionMiddleware
from google.oauth2.credentials import Credentials
import os
from src.logger import CustomLogger
from datetime import datetime, timezone
from fastapi.middleware.gzip import GZipMiddleware
import time
import gc

logger = CustomLogger()
CHUNK_DIR = os.path.join(os.path.dirname(__file__), "chunks")
MERGED_DIR = os.path.join(os.path.dirname(__file__), "merged_files")

def healthy_condition():
    output = {"healthy": True}
    return output

def healthy():
    return True

def sick():
    return False

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
# app.add_middleware(GZipMiddleware, minimum_size=1000)

is_gemini_enabled = os.environ.get("GEMINI_ENABLED", "False").lower()
in ("true", "1", "yes")
if is_gemini_enabled:
    add_routes(app,ChatVertexAI(), path="/vertexai")

app.add_api_route("/health", health([healthy_condition, healthy]))
```

```
app.add_middleware(SessionMiddleware, secret_key=os.urandom(24))


@app.post("/url/scan")
async def create_source_knowledge_graph_url(
    request: Request,
    uri=Form(None),
    userName=Form(None),
    password=Form(None),
    source_url=Form(None),
    database=Form(None),
    aws_access_key_id=Form(None),
    aws_secret_access_key=Form(None),
    wiki_query=Form(None),
    model=Form(None),
    gcs_bucket_name=Form(None),
    gcs_bucket_folder=Form(None),
    source_type=Form(None),
    gcs_project_id=Form(None),
    access_token=Form(None)
    ):

    try:
        if source_url is not None:
            source = source_url
        else:
            source = wiki_query

        graph = create_graph_database_connection(uri, userName,
password, database)
        if source_type == 's3 bucket' and aws_access_key_id and
aws_secret_access_key:
            lst_file_name,success_count,failed_count = await
asyncio.to_thread(create_source_node_graph_url_s3,graph, model,
source_url, aws_access_key_id, aws_secret_access_key, source_type
            )
        elif source_type == 'gcs bucket':
            lst_file_name,success_count,failed_count =
create_source_node_graph_url_gcs(graph, model, gcs_project_id,
gcs_bucket_name, gcs_bucket_folder,
source_type,Credentials(access_token)
            )
        elif source_type == 'web-url':
```

```
            lst_file_name,success_count,failed_count = await
asyncio.to_thread(create_source_node_graph_web_url,graph, model,
source_url, source_type
            )
        elif source_type == 'youtube':
            lst_file_name,success_count,failed_count = await
asyncio.to_thread(create_source_node_graph_url_youtube,graph, model,
source_url, source_type
            )
        elif source_type == 'Wikipedia':
            lst_file_name,success_count,failed_count = await
asyncio.to_thread(create_source_node_graph_url_wikipedia,graph, model,
wiki_query, source_type
            )
        else:
            return create_api_response('Failed',message='source_type is
other than accepted source')

        message = f"Source Node created successfully for source type:
{source_type} and source: {source}"
        josn_obj =
{'api_name':'url_scan','db_url':uri,'url_scanned_file':lst_file_name,
'source_url':source_url, 'wiki_query':wiki_query, 'logging_time':
formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        return
create_api_response("Success",message=message,success_count=success_cou
nt,failed_count=failed_count,file_name=lst_file_name)
    except Exception as e:
        error_message = str(e)
        message = f" Unable to create source node for source type:
{source_type} and source: {source}"
        logging.exception(f'Exception Stack trace:')
        return create_api_response('Failed',message=message +
error_message[:80],error=error_message,file_source=source_type)
    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'url/scan')

@app.post("/extract")
async def extract_knowledge_graph_from_file(
    uri=Form(None),
    userName=Form(None),
```

```
    password=Form(None),
    model=Form(None),
    database=Form(None),
    source_url=Form(None),
    aws_access_key_id=Form(None),
    aws_secret_access_key=Form(None),
    wiki_query=Form(None),
    max_sources=Form(None),
    gcs_project_id=Form(None),
    gcs_bucket_name=Form(None),
    gcs_bucket_folder=Form(None),
    gcs_blob_filename=Form(None),
    source_type=Form(None),
    file_name=Form(None),
    allowedNodes=Form(None),
    allowedRelationship=Form(None),
    language=Form(None),
    access_token=Form(None)
):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        graphDb_data_Access = graphDBdataAccess(graph)
        if source_type == 'local file':
            merged_file_path = os.path.join(MERGED_DIR,file_name)
            logging.info(f'File path:{merged_file_path}')
            result = await asyncio.to_thread(
                extract_graph_from_file_local_file, graph, model,
merged_file_path, file_name, allowedNodes, allowedRelationship, uri)

        elif source_type == 's3 bucket' and source_url:
            result = await asyncio.to_thread(
                extract_graph_from_file_s3, graph, model, source_url,
aws_access_key_id, aws_secret_access_key, allowedNodes,
allowedRelationship)

        elif source_type == 'web-url':
            result = await asyncio.to_thread(
                extract_graph_from_web_page, graph, model, source_url,
allowedNodes, allowedRelationship)

        elif source_type == 'youtube' and source_url:
            result = await asyncio.to_thread(
```

```
                extract_graph_from_file_youtube, graph, model,
source_url, allowedNodes, allowedRelationship)

        elif source_type == 'Wikipedia' and wiki_query:
            result = await asyncio.to_thread(
                extract_graph_from_file_Wikipedia, graph, model,
wiki_query, max_sources, language, allowedNodes, allowedRelationship)

        elif source_type == 'gcs bucket' and gcs_bucket_name:
            result = await asyncio.to_thread(
                extract_graph_from_file_gcs, graph, model,
gcs_project_id, gcs_bucket_name, gcs_bucket_folder, gcs_blob_filename,
access_token, allowedNodes, allowedRelationship)
        else:
            return create_api_response('Failed',message='source_type is
other than accepted source')
        if result is not None:
            result['db_url'] = uri
            result['api_name'] = 'extract'
            result['source_url'] = source_url
            result['wiki_query'] = wiki_query
            result['source_type'] = source_type
            result['logging_time'] =
formatted_time(datetime.now(timezone.utc))
        logger.log_struct(result)
        return create_api_response('Success', data=result, file_source=
source_type)
    except Exception as e:
        message=f"Failed To Process File:{file_name} or LLM Unable To
Parse Content "
        error_message = str(e)
        graphDb_data_Access.update_exception_db(file_name,error_message)
        gcs_file_cache = os.environ.get('GCS_FILE_CACHE')
        if source_type == 'local file':
            if gcs_file_cache == 'True':
                folder_name =
create_gcs_bucket_folder_name_hashed(uri,file_name)
                copy_failed_file(BUCKET_UPLOAD, BUCKET_FAILED_FILE,
folder_name, file_name)
                time.sleep(5)
                delete_file_from_gcs(BUCKET_UPLOAD,folder_name,file_name)
            else:
                logging.info(f'Deleted File Path: {merged_file_path} and
Deleted File Name : {file_name}')
```

```
                delete_uploaded_local_file(merged_file_path,file_name)
        josn_obj = {'message':message,'error_message':error_message,
'file_name': file_name,'status':'Failed','db_url':uri,'failed_count':1,
'source_type': source_type, 'source_url':source_url,
'wiki_query':wiki_query, 'logging_time':
formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        logging.exception(f'File Failed in extraction: {josn_obj}')
        return create_api_response('Failed', message=message +
error_message[:100], error=error_message, file_name = file_name)
    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'extract')

@app.get("/sources_list")
async def get_source_list(uri:str, userName:str, password:str,
database:str=None):
    """
    Calls 'get_source_list_from_graph' which returns list of sources
which already exist in databse
    """
    try:
        decoded_password = decode_password(password)
        if " " in uri:
            uri = uri.replace(" ","+")
        result = await
asyncio.to_thread(get_source_list_from_graph,uri,userName,decoded_passw
ord,database)
        josn_obj = {'api_name':'sources_list','db_url':uri,
'logging_time': formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        return create_api_response("Success",data=result)
    except Exception as e:
        job_status = "Failed"
        message="Unable to fetch source list"
        error_message = str(e)
        logging.exception(f'Exception:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)

@app.post("/post_processing")
async def post_processing(uri=Form(None), userName=Form(None),
password=Form(None), database=Form(None), tasks=Form(None)):
```

```
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        tasks = set(map(str.strip, json.loads(tasks)))

        if "update_similarity_graph" in tasks:
            await asyncio.to_thread(update_graph, graph)
            josn_obj = {'api_name':
'post_processing/update_similarity_graph', 'db_url': uri,
'logging_time': formatted_time(datetime.now(timezone.utc))}
            logger.log_struct(josn_obj)
            logging.info(f'Updated KNN Graph')
        if "create_fulltext_index" in tasks:
            await asyncio.to_thread(create_fulltext, uri=uri,
username=userName, password=password, database=database)
            josn_obj = {'api_name':
'post_processing/create_fulltext_index', 'db_url': uri, 'logging_time':
formatted_time(datetime.now(timezone.utc))}
            logger.log_struct(josn_obj)
            logging.info(f'Full Text index created')
        if os.environ.get('ENTITY_EMBEDDING','False').upper()=="TRUE"
and "create_entity_embedding" in tasks:
            await asyncio.to_thread(create_entity_embedding, graph)
            josn_obj = {'api_name':
'post_processing/create_entity_embedding', 'db_url': uri,
'logging_time': formatted_time(datetime.now(timezone.utc))}
            logger.log_struct(josn_obj)
            logging.info(f'Entity Embeddings created')
        return create_api_response('Success', message='All tasks
completed successfully')

    except Exception as e:
        job_status = "Failed"
        error_message = str(e)
        message = f"Unable to complete tasks"
        logging.exception(f'Exception in post_processing tasks:
{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)

    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'post_processing')
```

```
@app.post("/chat_bot")
async def chat_bot(uri=Form(None),model=Form(None),userName=Form(None),
password=Form(None), database=Form(None),question=Form(None),
document_names=Form(None),session_id=Form(None),mode=Form(None)):
    logging.info(f"QA_RAG called at {datetime.now()}")
    qa_rag_start_time = time.time()
    try:
        if mode == "graph":
            graph =
Neo4jGraph( url=uri,username=userName,password=password,database=databa
se,sanitize = True, refresh_schema=True)
        else:
            graph = create_graph_database_connection(uri, userName,
password, database)
        result = await
asyncio.to_thread(QA_RAG,graph=graph,model=model,question=question,docu
ment_names=document_names,session_id=session_id,mode=mode)

        total_call_time = time.time() - qa_rag_start_time
        logging.info(f"Total Response time is  {total_call_time:.2f}
seconds")
        result["info"]["response_time"] = round(total_call_time, 2)

        josn_obj =
{'api_name':'chat_bot','db_url':uri,'session_id':session_id,
'logging_time': formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        return create_api_response('Success',data=result)
    except Exception as e:
        job_status = "Failed"
        message="Unable to get chat response"
        error_message = str(e)
        logging.exception(f'Exception in chat bot:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        gc.collect()

@app.post("/chunk_entities")
async def chunk_entities(uri=Form(None),userName=Form(None),
password=Form(None), chunk_ids=Form(None)):
    try:
```

```
        logging.info(f"URI: {uri}, Username: {userName}, chunk_ids:
{chunk_ids}")
        result = await
asyncio.to_thread(get_entities_from_chunkids,uri=uri,
username=userName, password=password, chunk_ids=chunk_ids)
        josn_obj = {'api_name':'chunk_entities','db_url':uri,
'logging_time': formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        return create_api_response('Success',data=result)
    except Exception as e:
        job_status = "Failed"
        message="Unable to extract entities from chunk ids"
        error_message = str(e)
        logging.exception(f'Exception in chat bot:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        gc.collect()

@app.post("/graph_query")
async def graph_query(
    uri: str = Form(None),
    userName: str = Form(None),
    password: str = Form(None),
    document_names: str = Form(None),
):
    try:
        print(document_names)
        result = await asyncio.to_thread(
            get_graph_results,
            uri=uri,
            username=userName,
            password=password,
            document_names=document_names
        )
        josn_obj =
{'api_name':'graph_query','db_url':uri,'document_names':document_names,
'logging_time': formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        return create_api_response('Success', data=result)
    except Exception as e:
        job_status = "Failed"
        message = "Unable to get graph query response"
        error_message = str(e)
```

```
        logging.exception(f'Exception in graph query: {error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        gc.collect()


@app.post("/clear_chat_bot")
async def clear_chat_bot(uri=Form(None),userName=Form(None),
password=Form(None), database=Form(None), session_id=Form(None)):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        result = await
asyncio.to_thread(clear_chat_history,graph=graph,session_id=session_id)
        return create_api_response('Success',data=result)
    except Exception as e:
        job_status = "Failed"
        message="Unable to clear chat History"
        error_message = str(e)
        logging.exception(f'Exception in chat bot:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'clear_chat_bot')

@app.post("/connect")
async def connect(uri=Form(None), userName=Form(None),
password=Form(None), database=Form(None)):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        result = await asyncio.to_thread(connection_check, graph)
        josn_obj = {'api_name':'connect','db_url':uri,'status':result,
'count':1, 'logging_time': formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        return create_api_response('Success',message=result)
    except Exception as e:
        job_status = "Failed"
        message="Connection failed to connect Neo4j database"
        error_message = str(e)
```

```
        logging.exception(f'Connection failed to connect Neo4j
database:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)

@app.post("/upload")
async def upload_large_file_into_chunks(file:UploadFile = File(...),
chunkNumber=Form(None), totalChunks=Form(None),
                                        originalname=Form(None),
model=Form(None), uri=Form(None), userName=Form(None),
                                        password=Form(None),
database=Form(None)):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        result = await asyncio.to_thread(upload_file, graph, model,
file, chunkNumber, totalChunks, originalname, uri, CHUNK_DIR,
MERGED_DIR)
        josn_obj = {'api_name':'upload','db_url':uri, 'logging_time':
formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        if int(chunkNumber) == int(totalChunks):
            return create_api_response('Success',data=result,
message='Source Node Created Successfully')
        else:
            return create_api_response('Success', message=result)
    except Exception as e:
        # job_status = "Failed"
        message="Unable to upload large file into chunks. "
        error_message = str(e)
        logging.info(message)
        logging.exception(f'Exception:{error_message}')
        return create_api_response('Failed', message=message +
error_message[:100], error=error_message, file_name = originalname)
    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'upload')

@app.post("/schema")
async def get_structured_schema(uri=Form(None), userName=Form(None),
password=Form(None), database=Form(None)):
    try:
```

```
        graph = create_graph_database_connection(uri, userName,
password, database)
        result = await asyncio.to_thread(get_labels_and_relationtypes,
graph)
        logging.info(f'Schema result from DB: {result}')
        josn_obj = {'api_name':'schema','db_url':uri, 'logging_time':
formatted_time(datetime.now(timezone.utc))}
        logger.log_struct(josn_obj)
        return create_api_response('Success', data=result)
    except Exception as e:
        message="Unable to get the labels and relationtypes from neo4j
database"
        error_message = str(e)
        logging.info(message)
        logging.exception(f'Exception:{error_message}')
        return create_api_response("Failed", message=message,
error=error_message)
    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'schema')

def decode_password(pwd):
    sample_string_bytes = base64.b64decode(pwd)
    decoded_password = sample_string_bytes.decode("utf-8")
    return decoded_password

@app.get("/update_extract_status/{file_name}")
async def update_extract_status(request:Request, file_name, url,
userName, password, database):
    async def generate():
        status = ''
        decoded_password = decode_password(password)
        uri = url
        if " " in url:
            uri= url.replace(" ","+")
        while True:
            if await request.is_disconnected():
                logging.info("Request disconnected")
                break
            #get the current status of document node
            graph = create_graph_database_connection(uri, userName,
decoded_password, database)
            graphDb_data_Access = graphDBdataAccess(graph)
```

```
            result =
graphDb_data_Access.get_current_status_document_node(file_name)
            if result is not None:
                status = json.dumps({'fileName':file_name,
                'status':result[0]['Status'],
                'processingTime':result[0]['processingTime'],
                'nodeCount':result[0]['nodeCount'],
                'relationshipCount':result[0]['relationshipCount'],
                'model':result[0]['model'],
                'total_chunks':result[0]['total_chunks'],
                'total_pages':result[0]['total_pages'],
                'fileSize':result[0]['fileSize'],
                'processed_chunk':result[0]['processed_chunk'],
                'fileSource':result[0]['fileSource']
                })
            else:
                status = json.dumps({'fileName':file_name,
'status':'Failed'})
            yield status

    return EventSourceResponse(generate(),ping=60)

@app.post("/delete_document_and_entities")
async def delete_document_and_entities(uri=Form(),
                                       userName=Form(),
                                       password=Form(),
                                       database=Form(),
                                       filenames=Form(),
                                       source_types=Form(),
                                       deleteEntities=Form()):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        graphDb_data_Access = graphDBdataAccess(graph)
        result, files_list_size = await
asyncio.to_thread(graphDb_data_Access.delete_file_from_graph,
filenames, source_types, deleteEntities, MERGED_DIR, uri)
        entities_count = result[0]['deletedEntities'] if
'deletedEntities' in result[0] else 0
        message = f"Deleted {files_list_size} documents with
{entities_count} entities from database"
        josn_obj =
{'api_name':'delete_document_and_entities','db_url':uri,
'logging_time': formatted_time(datetime.now(timezone.utc))}
```

```
        logger.log_struct(josn_obj)
        return create_api_response('Success',message=message)
    except Exception as e:
        job_status = "Failed"
        message=f"Unable to delete document {filenames}"
        error_message = str(e)
        logging.exception(f'{message}:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'delete_document_and_entities')

@app.get('/document_status/{file_name}')
async def get_document_status(file_name, url, userName, password,
database):
    decoded_password = decode_password(password)

    try:
        if " " in url:
            uri= url.replace(" ","+")
        else:
            uri=url
        graph = create_graph_database_connection(uri, userName,
decoded_password, database)
        graphDb_data_Access = graphDBdataAccess(graph)
        result =
graphDb_data_Access.get_current_status_document_node(file_name)
        if result is not None:
            status = {'fileName':file_name,
                'status':result[0]['Status'],
                'processingTime':result[0]['processingTime'],
                'nodeCount':result[0]['nodeCount'],
                'relationshipCount':result[0]['relationshipCount'],
                'model':result[0]['model'],
                'total_chunks':result[0]['total_chunks'],
                'total_pages':result[0]['total_pages'],
                'fileSize':result[0]['fileSize'],
                'processed_chunk':result[0]['processed_chunk'],
                'fileSource':result[0]['fileSource']
                }
        else:
            status = {'fileName':file_name, 'status':'Failed'}
```

```
        return
create_api_response('Success',message="",file_name=status)
    except Exception as e:
        message=f"Unable to get the document status"
        error_message = str(e)
        logging.exception(f'{message}:{error_message}')
        return create_api_response('Failed',message=message)

@app.post("/cancelled_job")
async def cancelled_job(uri=Form(None), userName=Form(None),
password=Form(None), database=Form(None), filenames=Form(None),
source_types=Form(None)):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        result = manually_cancelled_job(graph,filenames, source_types,
MERGED_DIR, uri)

        return create_api_response('Success',message=result)
    except Exception as e:
        job_status = "Failed"
        message="Unable to cancelled the running job"
        error_message = str(e)
        logging.exception(f'Exception in cancelling the running
job:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        gc.collect()
        if graph is not None:
            close_db_connection(graph, 'cancelled_job')

@app.post("/populate_graph_schema")
async def populate_graph_schema(input_text=Form(None),
model=Form(None), is_schema_description_checked=Form(None)):
    try:
        result = populate_graph_schema_from_text(input_text, model,
is_schema_description_checked)
        return create_api_response('Success',data=result)
    except Exception as e:
        job_status = "Failed"
        message="Unable to get the schema from text"
        error_message = str(e)
```

```
        logging.exception(f'Exception in getting the schema from
text:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        gc.collect()

@app.post("/get_unconnected_nodes_list")
async def get_unconnected_nodes_list(uri=Form(), userName=Form(),
password=Form(), database=Form()):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        graphDb_data_Access = graphDBdataAccess(graph)
        nodes_list, total_nodes =
graphDb_data_Access.list_unconnected_nodes()
        return
create_api_response('Success',data=nodes_list,message=total_nodes)
    except Exception as e:
        job_status = "Failed"
        message="Unable to get the list of unconnected nodes"
        error_message = str(e)
        logging.exception(f'Exception in getting list of unconnected
nodes:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        if graph is not None:
            close_db_connection(graph,"get_unconnected_nodes_list")
        gc.collect()

@app.post("/delete_unconnected_nodes")
async def get_unconnected_nodes_list(uri=Form(), userName=Form(),
password=Form(), database=Form(),unconnected_entities_list=Form()):
    try:
        graph = create_graph_database_connection(uri, userName,
password, database)
        graphDb_data_Access = graphDBdataAccess(graph)
        result =
graphDb_data_Access.delete_unconnected_nodes(unconnected_entities_list)
        return
create_api_response('Success',data=result,message="Unconnected entities
delete successfully")
    except Exception as e:
```

```
        job_status = "Failed"
        message="Unable to delete the unconnected nodes"
        error_message = str(e)
        logging.exception(f'Exception in delete the unconnected
nodes:{error_message}')
        return create_api_response(job_status, message=message,
error=error_message)
    finally:
        if graph is not None:
            close_db_connection(graph,"delete_unconnected_nodes")
        gc.collect()


@app.post("/doubao/v1/chat/completions")
def generate_text(chat_request: Annotated[ChatRequest, Body()])->Dict:
    client = Ark(
        base_url="https://ark.cn-beijing.volces.com/api/v3",
        api_key=os.environ["DOUBAO_API_KEY"],
    )
    response = client.chat.completions.create(
        model=chat_request.model,
        messages=[m.model_dump() for m in chat_request.messages],
        frequency_penalty=chat_request.frequency_penalty,
        logit_bias=chat_request.logit_bias,
        logprobs=chat_request.logprobs,
        max_tokens=chat_request.max_tokens,
        presence_penalty=chat_request.presence_penalty,
        stop=chat_request.stop,
        stream=chat_request.stream,
        stream_options=chat_request.stream_options,
        tools=chat_request.tools,
        top_logprobs=chat_request.top_logprobs,
        top_p=chat_request.top_p,
        temperature=chat_request.temperature,
        user=chat_request.user,
    )

    return response.dict()

if __name__ == "__main__":
    # uvicorn.run(app, port=int(os.environ['TCP_PORT']))
    uvicorn.run(app, port=int(os.environ.get('TCP_PORT', '8080')))
```